

Phase 3: Iterative Insertion, Creating a BST from a List, and In-order Verification

1. Where You Are Now

You have already learned two important foundations:

Phase 1: Binary tree basics

You learned that a binary tree is made of nodes. Each node stores:

```
struct Node {  
    struct Node *lchild;  
    int data;  
    struct Node *rchild;  
};
```

You also learned:

- `root` is the top node.
- `lchild` points to the left child.
- `rchild` points to the right child.
- `NULL` means no node exists there.
- Recursive functions often stop when they reach `NULL`.
- `count()` visits all nodes.
- `height()` finds the longest downward path.

Phase 2: BST searching

You then learned that a Binary Search Tree, or BST, follows this rule:

```
left values < node value < right values
```

When searching:

```
key == current data  -> found  
key < current data   -> go left  
key > current data   -> go right  
NULL                 -> not found
```

The most important Phase 2 idea is that search follows only one path, not the whole tree.

2. The Big Idea of Phase 3

Phase 3 teaches insertion.

Insertion is almost the same as searching.

To insert a value:

1. Start at the root.
2. Compare the new value with the current node.
3. Move left or right using the BST rule.
4. If the value already exists, stop.
5. If you reach `NULL`, that empty place is where the new node belongs.

The only new difficulty is this:

When the search pointer reaches `NULL`, we have lost the parent node unless we saved it.

So we use two pointers:

```
t = current/explorer pointer  
r = parent/tail/follower pointer
```

`t` moves downward.

`r` stays one step behind `t`.

When `t` becomes `NULL`, `r` still points to the parent where the new node must be attached.

3. Why We Need the Tail Pointer `r`

Suppose we want to insert `38`.

Imagine the path is:

```
Root -> 40 -> 35 -> NULL
```

At the end:

```
t == NULL
```

But we cannot attach a new node to `NULL` directly.

We need to know the parent node.

That parent is 35.

So before every movement, we write:

```
r = t;
```

Then we move t:

```
if (key < t->data)
    t = t->lchild;
else
    t = t->rchild;
```

This means:

```
r remembers the last real node.
t explores the next child.
```

4. Iterative Insertion Logic

The algorithm is:

```
If the tree is empty:
    create the root node
    stop

Otherwise:
    t starts at root
    r starts as NULL

    while t is not NULL:
        r = t
        if key already exists:
            stop
        else if key is smaller:
            move t left
        else:
            move t right

    create a new node
```

```
if key is smaller than r->data:
    attach new node as r->lchild
else:
    attach new node as r->rchild
```

5. Code for Creating a New Node

```
struct Node* createNode(int key) {
    struct Node *p = (struct Node*)malloc(sizeof(struct Node));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        exit(1);
    }

    p->data = key;
    p->lchild = NULL;
    p->rchild = NULL;

    return p;
}
```

This creates one node, stores the key, and sets both child pointers to `NULL`.

6. Full Iterative Insert Function

```
struct Node *root = NULL;

void insert(int key) {
    struct Node *t = root;
    struct Node *r = NULL;
    struct Node *p;

    if (root == NULL) {
        root = createNode(key);
        return;
    }

    while (t != NULL) {
        r = t;

        if (key == t->data) {
            return;
        }
    }
}
```

```

        } else if (key < t->data) {
            t = t->lchild;
        } else {
            t = t->rchild;
        }
    }

    p = createNode(key);

    if (key < r->data) {
        r->lchild = p;
    } else {
        r->rchild = p;
    }
}

```

7. Dry Run: Inserting 38

Assume the search path is:

Root -> 40 -> 35 -> NULL

Step	t points to	r becomes	Comparison	Next move
1	Root	Root	38 is larger than Root	move right
2	40	40	38 is smaller than 40	move left
3	35	35	38 is larger than 35	move right
4	NULL	35	stop	attach 38 as right child of 35

At the end:

```

r = 35
t = NULL

```

Since:

38 > 35

we attach 38 as the right child of 35.

8. Creating a BST from a List

To build a whole BST, insert each value one by one.

Example list:

9, 15, 5, 20, 16, 8, 12, 3, 6

Insertion order matters.

Step-by-step result

Inserted key	Path followed	Attachment
9	tree is empty	root
15	9 -> right	right child of 9
5	9 -> left	left child of 9
20	9 -> right, 15 -> right	right child of 15
16	9 -> right, 15 -> right, 20 -> left	left child of 20
8	9 -> left, 5 -> right	right child of 5
12	9 -> right, 15 -> left	left child of 15
3	9 -> left, 5 -> left	left child of 5
6	9 -> left, 5 -> right, 8 -> left	left child of 8

The final tree is:

```
      9
     /
    5  15
   / \ /
  3  8 12 20
   /   /
  6   16
```

9. In-order Traversal

In-order traversal means:

left subtree -> current node -> right subtree

Code:

```
void inorder(struct Node *p) {  
    if (p != NULL) {  
        inorder(p->lchild);  
        printf("%d ", p->data);  
        inorder(p->rchild);  
    }  
}
```

For a valid BST, in-order traversal prints the values in sorted order.

For the list:

9, 15, 5, 20, 16, 8, 12, 3, 6

The in-order output is:

3 5 6 8 9 12 15 16 20

That sorted output is evidence that the BST property is correct.

10. Complete Phase 3 C Program

```
#include <stdio.h>  
#include <stdlib.h>  
  
struct Node {  
    struct Node *lchild;  
    int data;  
    struct Node *rchild;  
};  
  
struct Node *root = NULL;  
  
struct Node* createNode(int key) {  
    struct Node *p = (struct Node*)malloc(sizeof(struct Node));  
  
    if (p == NULL) {
```

```

        printf("Memory allocation failed\n");
        exit(1);
    }

    p->data = key;
    p->lchild = NULL;
    p->rchild = NULL;

    return p;
}

void insert(int key) {
    struct Node *t = root;
    struct Node *r = NULL;
    struct Node *p;

    if (root == NULL) {
        root = createNode(key);
        return;
    }

    while (t != NULL) {
        r = t;

        if (key == t->data) {
            return;
        } else if (key < t->data) {
            t = t->lchild;
        } else {
            t = t->rchild;
        }
    }

    p = createNode(key);

    if (key < r->data) {
        r->lchild = p;
    } else {
        r->rchild = p;
    }
}

void inorder(struct Node *p) {
    if (p != NULL) {
        inorder(p->lchild);
        printf("%d ", p->data);
        inorder(p->rchild);
    }
}

```



```

}

struct Node* search(int key) {
    struct Node *t = root;

    while (t != NULL) {
        if (key == t->data) {
            return t;
        } else if (key < t->data) {
            t = t->lchild;
        } else {
            t = t->rchild;
        }
    }

    return NULL;
}

int main() {
    int keys[] = {9, 15, 5, 20, 16, 8, 12, 3, 6};
    int n = sizeof(keys) / sizeof(keys[0]);

    for (int i = 0; i < n; i++) {
        insert(keys[i]);
    }

    printf("In-order traversal: ");
    inorder(root);
    printf("\n");

    printf("Search 16: %s\n", search(16) != NULL ? "found" : "not found");
    printf("Search 99: %s\n", search(99) != NULL ? "found" : "not found");

    return 0;
}

```

Expected output:

```

In-order traversal: 3 5 6 8 9 12 15 16 20
Search 16: found
Search 99: not found

```

11. Complexity

Insert one key

General time:

$O(h)$

If the tree is balanced:

$O(\log n)$

If the tree is skewed:

$O(n)$

Iterative insertion uses:

$O(1)$ extra space

because it only uses a few pointers.

Insert many keys

If you insert n keys one by one:

- Balanced case: $O(n \log n)$
- Skewed case: $O(n^2)$

In-order traversal

In-order traversal visits every node once:

$O(n)$

Its recursive stack space is:

$O(h)$

12. Common Beginner Mistakes

Mistake 1: Not handling an empty tree

If `root == NULL`, the first inserted node must become the root.

Correct:

```
if (root == NULL) {  
    root = createNode(key);  
    return;  
}
```

Mistake 2: Losing the parent pointer

When `t` becomes `NULL`, you cannot use `t` to link the new node.

Use `r` to remember the parent.

Mistake 3: Inserting duplicates

These notes use a standard BST that ignores duplicates.

Correct:

```
if (key == t->data) {  
    return;  
}
```

Mistake 4: Creating the node too early

Do not allocate a new node before checking whether the key already exists.

Search first. Allocate only after you find the correct `NULL` position.

Mistake 5: Thinking in-order is sorted for every binary tree

In-order traversal prints sorted values only if the tree is a valid BST.

13. Final Mental Model

Think of insertion like this:

Search for the key.
If you find it, do nothing.
If you fall into NULL, insert there.
Use r to remember the parent of that NULL spot.

The heart of Phase 3 is:

Insertion = search + parent pointer + new node linking